

Modularização de programas

Headers, múltiplos arquivos e bibliotecas

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

`andreyoshiaki@dcomp.ufs.br`

Separar *o que* um módulo oferece do *como* ele faz



O header (.h) como contrato — e as guardas de inclusão



De .c a executável: pré-processador, compilador, ligador



Um Makefile que recompila só o que mudou



Bibliotecas: estática (.a) e compartilhada (.so)

Onde estamos

O `notas.c` da aula passada tem 68 linhas e faz três coisas diferentes: cuida da matriz, calcula estatísticas e imprime o relatório.

Some mais duas funcionalidades e ele passa de 300.

O que dói quando ele cresce

Reuso: para usar `cria_matriz` em outro programa, você copia e cola.

Recompilação: mudou uma linha, recompila tudo.

Trabalho em equipe: dois colegas, um arquivo.

A pergunta de hoje

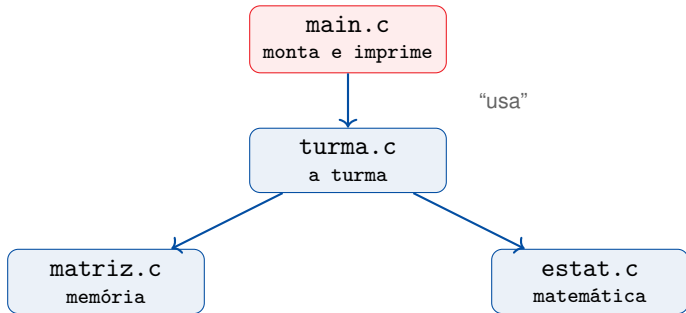
Como quebrar um programa em pedaços que compilam separado — e continuam formando um executável só?

O programa de hoje

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4
5  double **cria_matriz(int nl, int nc)    { /* ... */ }
6  void libera_matriz(double **m, int nl)  { /* ... */ }
7
8  double media(const double *v, int n)    { /* ... */ }
9  double desvio_padrao(const double *v, int n) { /* ... */ }
10
11 int main(void) { /* monta a turma, imprime media e desvio */ }
```

Três assuntos convivendo: **memória**, **matemática** e **apresentação**. Cada um muda por um motivo diferente.

A quebra: quatro arquivos, três responsabilidades



A seta vai de quem *usa* para quem *oferece*. Repare que `matriz.c` e `estat.c` não sabem que a turma existe — por isso servem para qualquer outro programa.

matriz.h — o que existe

```
#ifndef MATRIZ_H
#define MATRIZ_H

double **cria_matriz(int nl, int nc);
void libera_matriz(double **m, int nl);

#endif /* MATRIZ_H */
```

matriz.c — como funciona

```
#include <stdlib.h>
#include "matriz.h"

double **cria_matriz(int nl, int nc)
{
    double **m = malloc(nl * sizeof(double *));
    /* ... o mesmo código da aula passada ... */
    return m;
}
```

Sete linhas contra sessenta: o header cabe na tela, e é ele que o resto do programa lê.

estat.h

```
#ifndef ESTAT_H
#define ESTAT_H

double media(const double *v, int n);
double desvio_padrao(const double *v, int n);

#endif /* ESTAT_H */
```

estat.c

```
#include <math.h>          /* sqrt */
#include "estat.h"

double desvio_padrao(const double *v, int n)
{
    double mu = media(v, n), s = 0.0;
    /* ... */
    return n > 0 ? sqrt(s / n) : 0.0;
}
```

Guarde o `sqrt`: ele vai reaparecer no fim da aula, quando falarmos da biblioteca padrão.

turma.h: o header que também define um tipo



```
1  #ifndef TURMA_H
2  #define TURMA_H
3
4  typedef struct {
5      double **notas;
6      int  nalunos;
7      int  navaliacoes;
8  } Turma;
9
10 Turma *turma_cria(int nalunos, int navaliacoes);
11 void turma_libera(Turma *t);
12 void turma_relatorio(const Turma *t);
13
14 #endif /* TURMA_H */
```

```
1  #include <stdio.h>
2  #include "turma.h"
3  #include "matriz.h"
4  #include "estat.h"
5  static void cabecalho(void)           /* so turma.c enxerga */
6  {
7      printf("%-8s %8s %8s\n", "aluno", "media", "desvio");
8  }
9
10 void turma_relatorio(const Turma *t)
11 {
12     cabecalho();
13     for (int i = 0; i < t->nalunos; i++)
14         printf("%-8d %8.2f %8.2f\n", i, media(t->notas[i], t->navaliacoes),
15             desvio_padrao(t->notas[i], t->navaliacoes));
16 }
```

```
1  #include <stdio.h>
2  #include "turma.h"      /* nao inclui matriz.h nem estat.h */
3  int main(void)
4  {
5      double valores[3][4] = { { 8.0, 6.5, 9.0, 7.0 },
6                               { 5.0, 7.0, 4.0, 7.0 },
7                               { 9.0, 10.0, 8.5, 9.5 } };
8      Turma *t = turma_cria(3, 4);
9      if (t == NULL) { fprintf(stderr, "sem memoria\n"); return 1; }
10     for (int i = 0; i < t->nalunos; i++)
11         for (int j = 0; j < t->navaliacoes; j++)
12             t->notas[i][j] = valores[i][j];
13     turma_relatorio(t);
14     turma_libera(t);
15     return 0;
16 }
```

Antes de compilar, escreva sua resposta

1. `gcc -c matriz.c` produz o quê? Esse arquivo roda?
2. `main.c` não inclui `matriz.h`. Ainda assim o executável final vai conter `cria_matriz`?
3. O que acontece se você compilar `gcc matriz.c estat.c turma.c -o notas`, esquecendo `main.c`?
4. E se esquecer o `-lm` na hora de juntar tudo?

```
$ gcc -Wall -Wextra -c matriz.c estat.c turma.c main.c
$ ls *.o
estat.o  main.o  matriz.o  turma.o
$ gcc matriz.o estat.o turma.o main.o -o notas -lm
$ ./notas
aluno      media  desvio
0           7.62   0.96
1           5.75   1.30
2           9.25   0.56
```

A saída é a mesma da aula passada. O que mudou não foi o programa — foi o *caminho* até o executável.

Interface e implementação

Definição

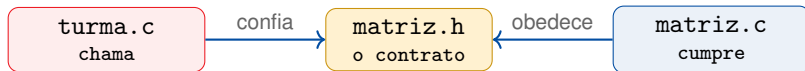
Declaração anuncia que um nome existe e qual é o seu tipo: `double media(const double *v, int n);`

Definição fornece o corpo — o código que será executado.

Intuição

A declaração é a ficha do cardápio: nome do prato, o que entra, o que sai. A definição é a cozinha. Quem faz o pedido só precisa do cardápio.

O header é um contrato entre dois lados



Os dois lados leem o *mesmo* arquivo. Por isso o compilador consegue avisar quando a chamada e a implementação discordam — desde que os dois incluam o header.

No header (.h)	No fonte (.c)
protótipos de funções públicas	corpo das funções
typedef e struct usados na interface	funções auxiliares <code>static</code>
enum e <code>#define</code> de constantes	variáveis do módulo (<code>static</code>)
<code>extern</code> de variáveis globais	a <i>definição</i> dessas globais

Regra prática

Se remover a linha do header quebra quem usa o módulo, ela pertence ao header. Se não quebra, ela é detalhe de implementação.

```
$ cat mini.c
#include "matriz.h"
int main(void) { return 0; }

$ gcc -E mini.c           # so o pre-processor
# 1 "mini.c"
# 1 "./matriz.h" 1
double **cria_matriz(int nl, int nc);
void libera_matriz(double **m, int nl);
# 2 "mini.c" 2
int main(void) { return 0; }
```

O pré-processador não entende C: ele troca a linha `#include` pelo conteúdo do arquivo. As linhas com `#` apenas registram de onde veio cada trecho. Com `<stdio.h>` no lugar, a saída passa de 500 linhas.

"aspas" ou <ângulos>?



```
#include "matriz.h"
```

Procura primeiro no diretório do arquivo que incluiu; depois nos diretórios do sistema.

Use para os *seus* headers.

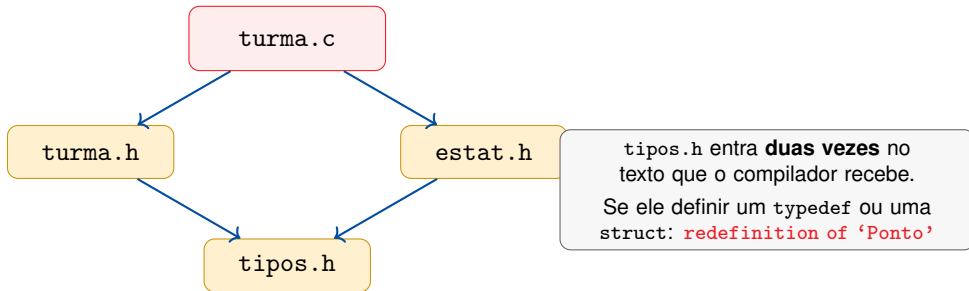
```
#include <stdio.h>
```

Procura só nos diretórios do sistema (e nos que você acrescentar com -I).

Use para a biblioteca padrão e para bibliotecas instaladas.

`gcc -Icaminho/include` acrescenta um diretório à busca — é assim que você usa headers que não estão ao lado do seu `.c`.

O problema: o mesmo header, duas vezes



```
#ifndef MATRIZ_H      /* se ainda nao definido
    */
#define MATRIZ_H      /* marque como definido
    */

double **cria_matriz(int nl, int nc);
void libera_matriz(double **m, int nl);

#endif /* MATRIZ_H */
```

Na segunda inclusão, `MATRIZ_H` já existe: tudo entre `#ifndef` e `#endif` é descartado.

O nome da macro precisa ser único no projeto — costume: `NOMEDOARQUIVO_H`.

`#pragma once` faz o mesmo em uma linha e é aceito por todos os compiladores atuais, mas não é parte do padrão C. Prefira as guardas quando o código precisar ser portátil.

O que aprendemos

- declarar anuncia; definir fornece o corpo — o header carrega as declarações;
- quem usa e quem implementa leem o mesmo `.h`: é isso que permite ao compilador conferir as chamadas;
- `#include` é substituição de texto, feita antes de o compilador ver qualquer C;
- toda `.h` sua começa com `#ifndef/#define` e termina com `#endif`.

De .c a executável

As quatro etapas escondidas atrás de `gcc`



Cada flag *para* o processo em um ponto. `gcc arquivo.c` sem flag nenhuma percorre as quatro etapas de uma vez — por isso parece um passo só.

O que o compilador enxerga ao ler

`turma.c`

O texto de `turma.c` depois dos `includes` — e mais nada. Ele *não* abre `matriz.c`.

Consequência

Sobre `cria_matriz` ele sabe só o que o header declarou: o tipo. Basta para gerar a chamada; não basta para saber onde o código está.

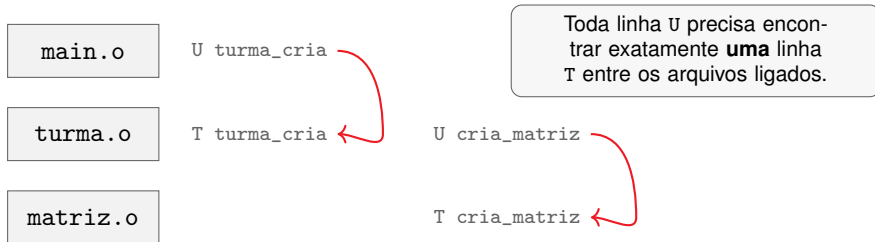
Quem resolve o resto

O **ligador**, depois, com todos os `.o` na mão. É por isso que erros de nome errado aparecem tão tarde.

```
$ nm turma.o
0000000000000000 T turma_cria          # T = definido aqui (publico)
0000000000000090 T turma_libera
00000000000000d0 T turma_relatorio
0000000000000160 t cabecalho          # t = definido, mas so daqui
                  U cria_matriz        # U = usado, definido em outro lugar
                  U desvio_padrao
                  U media
                  U printf
```

Um .o é código de máquina *mais* duas listas: o que ele oferece (T) e o que ele ainda deve (U).

O ligador casa dívidas com ofertas



Nenhuma linha U sobrando, nenhuma T repetida: é esse o fecho de contas que decide se o executável nasce.

```
$ gcc main.o -o notas
/usr/bin/ld: main.o: in function `main':
main.c:(.text+0x2a): undefined reference to `turma_cria'
collect2: error: ld returned 1 exit status
```

As três causas, em ordem de frequência

1. esqueceu um `.o` (ou um `.c`) na linha de comando;
2. escreveu o nome diferente no `.h` e no `.c` (`turma_cria` × `cria_turma`);
3. a função existe, mas está em uma biblioteca que você não pediu para ligar (`-lm`, por exemplo).

O que provoca

```
/* config.h -- ERRADO */  
int contador = 0; /* definicao! */
```

Incluído em dois .c, vira duas definições da mesma variável.

```
/usr/bin/ld: turma.o:(.bss+0x0): multiple definition of `contador';  
main.o:(.bss+0x0): first defined here
```

O jeito certo

```
/* config.h */  
extern int contador; /* so anuncia */  
  
/* config.c */  
int contador = 0; /* uma definicao */
```

```
static void cabecalho(void) { ... }    /* funcao so de turma.c */  
static int chamadas = 0;               /* variavel so de turma.c */
```

O que muda

O símbolo sai do .o como `t` minúsculo: o ligador não o oferece a ninguém.

Por que usar

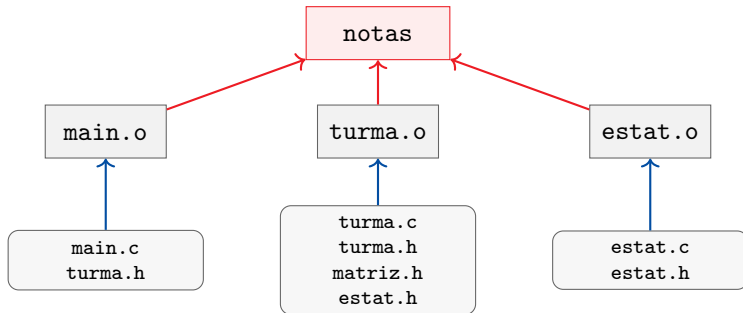
Dois módulos podem ter cada um a sua cabecalho sem colidir, e você deixa claro o que é detalhe interno.

Dentro de uma função, `static` quer dizer outra coisa: a variável sobrevive entre as chamadas. Mesma palavra, dois significados.

O que aprendemos

- `-c` para em `.o`; sem flag, `gcc` vai até o executável;
- cada `.c` é compilado sozinho — o compilador confia no header;
- o `.o` lista o que oferece (T) e o que deve (U); o ligador faz o casamento;
- `undefined reference` = faltou; `multiple definition` = sobrou; `static` esconde.

Automatizar: make



Mudou estat.c: refazer estat.o e religar. Mudou turma.h: main.o e turma.o caem junto. Quem decide isso pela data dos arquivos é o make.

```
alvo: pre-requisitos
<TAB>receita

estat.o: estat.c estat.h
        gcc -Wall -Wextra -c estat.c
```

A regra de ouro do `make`

Se o **alvo** não existe, ou se algum **pré-requisito** é mais recente que ele, execute a **receita**. Só isso.

A receita precisa começar com TAB de verdade — espaços dão `missing separator`. É o erro número um de quem começa.

```
CC      = gcc
CFLAGS  = -Wall -Wextra -g
LDLIBS  = -lm
OBJ      = main.o turma.o matriz.o estat.o

notas: $(OBJ)
    $(CC) $(OBJ) -o notas $(LDLIBS)

main.o:  main.c  turma.h
turma.o:  turma.c  turma.h matriz.h estat.h
matriz.o: matriz.c matriz.h
estat.o:  estat.c  estat.h

.PHONY: clean
clean:
    rm -f $(OBJ) notas
```

O que o `make` já sabe

Existe uma regra implícita para `%.o`: `%.c`,
que roda `$(CC) $(CFLAGS) -c`.

O que você acrescenta

Só a informação que o `make` não tem como
adivinhar: de quais *headers* cada objeto
depende.

`.PHONY`

`clean` não é um arquivo. Sem `.PHONY`: `clean`, se alguém criar um arquivo chamado `clean`, o `make` passa a dizer “`clean is up to date`” e não limpa mais nada.

```
$ make                                # primeira vez: compila tudo
gcc -Wall -Wextra -g -c -o main.o main.c
gcc -Wall -Wextra -g -c -o turma.o turma.c
gcc -Wall -Wextra -g -c -o matriz.o matriz.c
gcc -Wall -Wextra -g -c -o estat.o estat.c
gcc main.o turma.o matriz.o estat.o -o notas -lm

$ make                                # nada mudou
make: 'notas' is up to date.

$ touch estat.c && make                # so o que depende de estat.c
gcc -Wall -Wextra -g -c -o estat.o estat.c
gcc main.o turma.o matriz.o estat.o -o notas -lm
```

O que aprendemos

- o `make` compara datas: alvo mais velho que pré-requisito $\Rightarrow$ refaz;
- a receita vai depois de um TAB;
- listar os headers de cada objeto é o que faz o `make` acertar quando uma interface muda;
- `.PHONY` marca alvos que não são arquivos.

Bibliotecas

Definição

Uma **biblioteca** é um pacote de arquivos-objeto já compilados, distribuído junto com os headers que descrevem o que ela oferece.

Intuição

É o passo seguinte da modularização: em vez de entregar quatro `.o` soltos e torcer para ninguém esquecer um, você entrega *um* arquivo — e um `.h` dizendo o que tem dentro.

```
$ gcc -Wall -c matriz.c estat.c turma.c
$ ar rcs libnotas.a matriz.o estat.o turma.o      # r=inserir c=criar s=indice
$ ar t libnotas.a
matriz.o
estat.o
turma.o
$ gcc main.c -I. -L. -lnotas -lm -o notas
```

Os três flags da ligação

-I. onde procurar headers -L. onde procurar bibliotecas -lnotas qual usar — o nome do arquivo é *libnotas.a*.

O ligador copia só o que você usa



Se `estat.o` não fosse usado por ninguém, ele ficaria de fora. O executável final é autossuficiente: a `.a` pode ser apagada depois.

```
$ gcc -fPIC -c matriz.c estat.c turma.c          # codigo independente de posicao
$ gcc -shared -o libnotas.so matriz.o estat.o turma.o -lm
$ gcc main.c -I. -L. -lnotas -o notas
$ ./notas
./notas: error while loading shared libraries: libnotas.so:
cannot open shared object file: No such file or directory
$ LD_LIBRARY_PATH=. ./notas                      # agora sim
```

Ligou sem reclamar e mesmo assim falhou ao rodar: a `.so` não entra no executável, fica só o *nome* dela. Quem a procura, e não acha, é o carregador do sistema.

estática

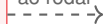
main.o
+ turma.o
+ matriz.o

tudo dentro;
roda sozinho

compartilhada

main.o
+ "preciso de
libnotas.so"

ao rodar



libnotas.so

o arquivo precisa estar
lá na hora da execução

	Estática (.a)	Compartilhada (.so)
Quando entra	na ligação	ao executar
Executável	maior, completo	menor, depende do arquivo
Corrigir um bug	religar tudo	trocar a .so
Dez programas	dez cópias no disco	uma cópia na memória
Risco	versão congelada	sumir/mudar sem avisar

`ldd notas` lista de quais .so um executável depende (no macOS, `otool -L`).

```
$ gcc main.o turma.o matriz.o estat.o -o notas
/usr/bin/ld: estat.o: undefined reference to symbol 'sqrt@@GLIBC_2.2.5'
collect2: error: ld returned 1 exit status
$ gcc main.o turma.o matriz.o estat.o -o notas -lm      # resolvido
```

`math.h` e `libm` são coisas distintas

O header traz a *declaração* de `sqrt`; o código está em `libm`. Incluir o header engana o compilador, não o ligador. Já `printf` não pede flag porque `libc` é ligada sempre.

Em alguns sistemas (macOS, glibc recente) a `libm` já vem junto e o erro não aparece. Escreva o `-lm` assim mesmo.

```
$ gcc -L. -lnotas main.c -o notas      # ERRADO: undefined reference  
$ gcc main.c -L. -lnotas -o notas     # certo
```

Por quê

O ligador lê a linha de comando da esquerda para a direita acumulando dívidas. Quando ele abriu `libnotas` no primeiro comando, ninguém ainda devia `turma_cria` — e ele seguiu adiante sem pegar nada.

Regra

Quem *usa* vem antes de quem *fornece*: primeiro seus arquivos, depois as bibliotecas.

O que aprendemos

- biblioteca = .o empacotados + headers que os descrevem;
- estática: `ar rcs libnome.a`, copiada para dentro do executável na ligação;
- compartilhada: `-fPIC -shared`, encontrada na hora de executar;
- `-I` headers, `-L` onde procurar, `-l` qual ligar — e as bibliotecas por último.

Os erros que o compilador não pega

Erro	Quando aparece	Sintoma
<code>#include "matriz.c"</code>	na ligação	<code>multiple definition</code>
Protótipo $\neq$ definição	ao executar	resultado sem sentido
Makefile sem o header	ao executar	lixo, ou queda sem motivo
Header sem guarda	ao compilar	<code>redefinition of...</code>

Os dois do meio são os caros: o programa compila, liga, roda — e mente.

```
#include "matriz.c"      /* funciona... ate voce compilar matriz.c tambem */
```

O que acontece

O corpo das funções é copiado para dentro do seu .c. Se `matriz.c` também for compilado e ligado, o ligador encontra `cria_matriz` duas vezes: `multiple definition`.

Como evitar

Header nunca tem corpo de função; .c nunca é incluído. Se está incluindo um .c para “funcionar de uma vez”, o que faltou foi passar os dois arquivos ao `gcc`.

estat.h

```
int media(const double *v, int n);
```

estat.c

```
double media(const double *v, int n)
{ ... }
```

Por que ninguém reclama

turma.c acredita no header e lê o retorno como `int`; estat.c devolve um `double`. Cada unidade de tradução está coerente consigo mesma, e o ligador só compara *nomes*.

A defesa

estat.c **inclui** estat.h. Aí o conflito passa a acontecer dentro de um arquivo só — e vira erro de compilação.

```
main.o: main.c          # faltou turma.h
```

O estrago

Você acrescenta um campo no meio da struct Turma. turma.o é feito com o novo desenho; main.o, não. Os dois passam a discordar de onde cada campo começa — e o make diz que está tudo em dia.

As saídas

make clean && make resolve na hora. Para não depender de memória, gcc -MMD -MP gera os arquivos .d com as dependências de header, e o Makefile os inclui.

O que aprendemos

- `.h` declara, `.c` define, e `.c` não se inclui;
- todo `.c` inclui o próprio `.h`: é a única forma de o compilador conferir o contrato;
- dependência de header esquecida no `Makefile` produz bugs que somem quando você recompila do zero — desconfie deles;
- compile sempre com `-Wall -Wextra`.

Mãos à obra

Tarefa 1

Crie o módulo `io.h/io.c` com `turma_le(Turma *t)`, que preenche as notas a partir do teclado, e tire essa responsabilidade do `main.c`. Acrescente as regras correspondentes ao `Makefile`.

Tarefa 2

Empacote `matriz.o` e `estat.o` em `libnotas.a` e ligue o programa contra ela. Depois apague os `.o` e recompile só o `main.c`: o executável ainda se forma?

Tarefa 3

Acrescente um campo no início da struct `Turma`, recompile só `turma.c` na mão e rode. Explique a saída — e depois conserte pelo `Makefile`.

Uma biblioteca de verdade, com tipo opaco

Reescreva o módulo da turma de modo que `turma.h` contenha apenas `typedef struct Turma Turma;` — sem os campos:

- a `struct` completa fica em `turma.c`; quem usa a biblioteca chega às notas por `turma_set` e `turma_get`;
- gere `libturma.a` e `libturma.so` dos mesmos fontes, com dois alvos no `Makefile`;
- escreva um cliente em *outro* diretório, compilado com `-I` e `-L`, sem copiar nenhum `.c`;
- explique por que agora o cliente não consegue depender do formato interno da `struct`.

O que mudou hoje

O programa é o mesmo e imprime o mesmo. O que ganhamos foi mexer em um pedaço sem tocar — nem recompilar — os outros.

O fio da aula

Header é contrato, `.o` é dívida e oferta, ligador é o casamenteiro, `make` é a memória das datas e biblioteca é o pacote pronto.

Para levar

- uma responsabilidade por módulo; o `.h` é a única porta;
- cada `.c` compila sozinho — o erro aparece na ligação;
- `-I`, `-L`, `-l`: onde estão os headers, onde estão as bibliotecas, quais usar;
- o bug mais difícil da aula é o que some depois de um `make clean`.

-  André Backes.
Linguagem C: completa e descomplicada.
Elsevier, Rio de Janeiro, 2013.
-  Randal E. Bryant and David R. O'Hallaron.
Computer Systems: A Programmer's Perspective.
Pearson, Boston, 3 edition, 2015.
Seção 3.9: alinhamento de dados e layout de structs.
-  Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Algoritmos: teoria e prática.
Elsevier, Rio de Janeiro, 3 edition, 2012.

-  **Paul Deitel and Harvey Deitel.**
C: como programar.
Pearson Prentice Hall, São Paulo, 6 edition, 2011.
-  **Ulrich Drepper.**
What Every Programmer Should Know About Memory.
Red Hat, Inc., 2007.
Seção 6.2: efeito da ordem de percurso de matrizes sobre a cache.
-  **Ulrich Drepper.**
How to write shared libraries.
Documento técnico, 2011.
Versão 4.1.2.



Paulo Feofiloff.

Algoritmos em linguagem C.

Elsevier, Rio de Janeiro, 2009.



Free Software Foundation.

Gnu coding standards.

Free Software Foundation.

Acesso em 2026.



Free Software Foundation.

Using the GNU compiler collection — options to request or suppress warnings.

Manual do GCC.

Acesso em 2026.



ISO/IEC.

Iso/iec 9899:2018 — information technology: Programming languages: C. Technical report, International Organization for Standardization, 2018.

Seção 6.7.2.1: structure and union specifiers.

-  Brian W. Kernighan and Rob Pike.
The Practice of Programming.
Addison-Wesley, Reading, MA, 1999.
Cap. 1, “Style”: nomes, expressões, consistência, números mágicos e comentários.
-  Brian W. Kernighan and Dennis M. Ritchie.
C: a linguagem de programação — padrão ANSI.
Campus, Rio de Janeiro, 1989.

-  K. N. King.
C Programming: A Modern Approach.
W. W. Norton, New York, 2 edition, 2008.
Capítulo 16: estruturas, uniões e enumerações.
-  Steve McConnell.
Code Complete: A Practical Handbook of Software Construction.
Microsoft Press, Redmond, WA, 2 edition, 2004.
Cap. 11 (nomes de variáveis), cap. 31 (layout e estilo) e cap. 32 (código autodocumentado).

-  Robert Mecklenburg.
Managing Projects with GNU Make.
O'Reilly Media, 3 edition, 2004.
-  Herbert Schildt.
C completo e total.
Makron Books, São Paulo, 3 edition, 1997.
-  Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov.
AddressSanitizer: A fast address sanity checker.
In *USENIX Annual Technical Conference (ATC)*, pages 309–318, 2012.
Ferramenta usada no tutorial para detectar vazamentos e uso após free.

-  Aaron M. Tenenbaum, Yedidyah Langsam, and Moshe J. Augenstein.
Estruturas de dados usando C.
Pearson Makron Books, São Paulo, 1995.
-  The Linux Kernel Developers.
Linux kernel coding style.
Documentação do kernel Linux, [Documentation/process/coding-style.rst](#).
Acesso em 2026.
-  The LLVM Project.
Clang-format style options.
Documentação do Clang.
Acesso em 2026.

-  Paul R. Wilson, Mark S. Johnstone, Michael Neely, and David Boles.
Dynamic storage allocation: A survey and critical review.
In *International Workshop on Memory Management (IWMM)*, pages 1–116.
Springer, 1995.
Panorama de estratégias de alocação dinâmica e fragmentação.
-  Nivio Ziviani.
Projeto de algoritmos: com implementações em Pascal e C.
Cengage Learning, São Paulo, 3 edition, 2011.